Linear Algebra and its Applications Assignment 4
Name: Arushi Marwaha
Roll Number: MDS202512
Course: MSc Data Science 1

Question 1(a)

Let $q_1, q_2 \in \mathbb{R}^3$ be orthonormal vectors and let $Q = [\, q_1 \; q_2 \,]$. Explain why the matrix QQ^T maps any vector $b \in \mathbb{R}^3$ to the closest vector in the subspace spanned by q_1 and q_2.

Solution

Let $q_1, q_2 \in \mathbb{R}^3$ be orthonormal vectors and define

$$Q = [\, q_1 \; q_2 \,] \in \mathbb{R}^{3\times 2}.$$

Then $Q^TQ = I_2$. Let $S = \text{span}\{q_1, q_2\} = \text{Col}(Q)$.

We show that for any $b \in \mathbb{R}^3$, the vector QQ^Tb is the closest vector in S to b.

Define

$$p := QQ^Tb \in S, \quad r := b - QQ^Tb, \quad b = p + r.$$

We show that $r \perp S$. Let $y \in S$. Then there exists $x \in \mathbb{R}^2$ such that $y = Qx$. Then

$$\langle r, y\rangle = \langle b - QQ^Tb, Qx\rangle = (Qx)^T(b - QQ^Tb) = x^TQ^Tb - x^TQ^TQQ^Tb.$$

Since $Q^TQ = I_2$, this becomes

$$x^TQ^Tb - x^T(I_2)Q^Tb = x^TQ^Tb - x^TQ^Tb = 0.$$

Thus $r \in S^\perp$.

Hence we have the orthogonal decomposition

$$b = QQ^Tb + (I - QQ^T)b,$$

where $QQ^Tb \in S$ and $(I - QQ^T)b \in S^\perp$.

Let $y \in S$. Then $y = Qx$ for some x. We write

$$b - y = (QQ^Tb - y) + (b - QQ^Tb).$$

Since $QQ^Tb - y \in S$ and $b - QQ^Tb \in S^\perp$, these two vectors are orthogonal. Therefore, by the Pythagorean Theorem:

$$\|b - y\|^2 = \|QQ^Tb - y\|^2 + \|b - QQ^Tb\|^2.$$

Since $\|QQ^Tb - y\|^2 \geq 0$, we get

$$\|b - y\|^2 \geq \|b - QQ^Tb\|^2.$$

Equality holds if and only if $\|QQ^Tb - y\|^2 = 0$, which implies $y = QQ^Tb$.

Thus QQ^Tb is the unique vector in S minimizing $\|b - y\|$. Therefore,

$$QQ^Tb$$

is the orthogonal projection of b onto $\text{span}\{q_1, q_2\}$, and hence the closest vector in that subspace. ■

—

Question 1(b)

Describe geometrically what the matrix $I - QQ^T$ does to a vector.

Solution

Let $S = \text{span}\{q_1, q_2\}$. Since q_1, q_2 are orthonormal, the matrix QQ^T is the orthogonal projection onto S.

For any $b \in \mathbb{R}^3$,

$$(I - QQ^T)b = b - QQ^T b.$$

Here, $QQ^T b \in S$ is the projection of b onto the subspace, so the vector $b - QQ^T b$ is orthogonal to S. Hence,

$$(I - QQ^T)b \in S^\perp.$$

Thus, we obtain the orthogonal decomposition

$$b = QQ^T b + (I - QQ^T)b,$$

where the two components lie in S and $S^\perp$, respectively.

Geometric interpretation:

The matrix $I - QQ^T$ maps a vector to its projection onto the orthogonal complement $S^\perp$. It extracts the component of the vector that lies perpendicular to the subspace spanned by q_1 and q_2. ■

—

Question 1(c)

Suppose $b = p + r$ where p lies in the column space of Q and r is orthogonal to it. Explain how $QQ^T b$ and $(I - QQ^T)b$ recover these components.

Solution

Let $S = \text{Col}(Q)$. Then $p \in S$ and $r \in S^\perp$.

Since QQ^T is the orthogonal projection onto S, we have:

$$QQ^T p = p \quad \text{and} \quad QQ^T r = 0.$$

Thus,

$$QQ^T b = QQ^T(p + r) = QQ^T p + QQ^T r = p + 0 = p.$$

Similarly,

$$(I - QQ^T)b = (I - QQ^T)(p + r) = (p + r) - QQ^T(p + r) = (p + r) - p = r.$$

The matrix QQ^T extracts the component of b in the column space of Q, while $(I - QQ^T)$ extracts the component orthogonal to it.

Question 2(a)

Let $Q \in \mathbb{R}^{n \times n}$ be an orthogonal matrix. Show that for any vector x,

$$\|Qx\|_2 = \|x\|_2.$$

Solution

Since Q is orthogonal, we have

$$Q^TQ = I.$$

For any $x \in \mathbb{R}^n$,

$$\|Qx\|_2^2 = (Qx)^T(Qx) = x^TQ^TQx = x^TIx = x^Tx = \|x\|_2^2.$$

Taking square roots on both sides,

$$\|Qx\|_2 = \|x\|_2.$$

— ■

Question 2(b)

Consider the least squares problem

$$\min_x \|Ax - b\|_2.$$

Explain why replacing A with Q^TA and b with Q^Tb does not change the minimizer.

Solution

Since Q is orthogonal, we have $Q^TQ = I$ and, from part (a), $\|Qy\|_2 = \|y\|_2$ for all y.

For any x,

$$\|Ax - b\|_2 = \|Q^T(Ax - b)\|_2.$$

Using linearity,

$$Q^T(Ax - b) = Q^TAx - Q^Tb.$$

Hence,

$$\|Ax - b\|_2 = \|Q^TAx - Q^Tb\|_2.$$

Therefore, the two minimization problems

$$\min_x \|Ax - b\|_2 \quad \text{and} \quad \min_x \|Q^TAx - Q^Tb\|_2$$

have identical objective values for every x, and thus the same minimizer.

Question 3(a)

Let

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Perform the classical Gram–Schmidt process to compute the QR decomposition $A = QR$.

Solution

Let

$$a_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad a_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

$$q_1 = \frac{a_1}{\|a_1\|}, \quad \|a_1\| = \sqrt{2} \quad \Rightarrow \quad q_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

$$r_{12} = q_1^T a_2 = \frac{1}{\sqrt{2}}.$$

$$v_2 = a_2 - r_{12} q_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{bmatrix}.$$

$$\|v_2\| = \sqrt{\frac{3}{2}}, \quad q_2 = \frac{v_2}{\|v_2\|} = \begin{bmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{bmatrix}.$$

Thus,

$$Q = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} \\ 0 & \frac{2}{\sqrt{6}} \end{bmatrix}, \quad R = \begin{bmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} \\ 0 & \sqrt{\frac{3}{2}} \end{bmatrix}.$$

■

—

Question 3(b)

Verify that $Q^T Q = I$.

Solution

Recall

$$q_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}, \quad q_2 = \begin{bmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{bmatrix}.$$

First computing $q_1^T q_1$,

$$q_1^T q_1 = \left(\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{1}{\sqrt{2}}\right)^2 + 0^2 = \frac{1}{2} + \frac{1}{2} + 0 = 1.$$

Now computing $q_2^T q_2$,

$$q_2^T q_2 = \left(\frac{1}{\sqrt{6}}\right)^2 + \left(-\frac{1}{\sqrt{6}}\right)^2 + \left(\frac{2}{\sqrt{6}}\right)^2 = \frac{1}{6} + \frac{1}{6} + \frac{4}{6} = \frac{6}{6} = 1.$$

Now computing $q_1^T q_2$,

$$\begin{aligned} q_1^T q_2 &= \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{6}} + \frac{1}{\sqrt{2}} \cdot \left(-\frac{1}{\sqrt{6}}\right) + 0 \cdot \frac{2}{\sqrt{6}} \\ &= \frac{1}{\sqrt{12}} - \frac{1}{\sqrt{12}} + 0 = 0. \end{aligned}$$

Thus,

$$Q^T Q = \begin{bmatrix} q_1^T q_1 & q_1^T q_2 \\ q_2^T q_1 & q_2^T q_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I.$$

■ —

Question 3 (c)

What geometric information about the columns of A is captured by the entries of R?

Solution

The matrix R encodes geometric relationships between the columns of A:

- $r_{11} = \|a_1\|$ gives the length of the first column.
- $r_{12} = q_1^T a_2$ gives the projection of a_2 onto q_1.
- $r_{22} = \|v_2\|$ gives the length of the component of a_2 orthogonal to q_1.

Thus, R not only records the magnitudes of the columns but also describes how each column decomposes into components along and orthogonal to the previously constructed orthonormal directions.

LAA_A4

March 22, 2026

1 Question 4

Consider the matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1+\epsilon \\ 1 & 1+\epsilon & 1 \end{bmatrix}, \quad \epsilon = 10^{-8}.$$

(a) Implement the classical Gram–Schmidt algorithm.

(b) Implement the modified Gram–Schmidt algorithm.

(c) Compute the matrices Q_{CGS} and Q_{MGS}.

(d) Measure the orthogonality error

$$\|I - Q^T Q\|_F$$

for both methods.

(e) Compare the results and briefly explain which method preserves orthogonality better.

[37]:
```python
import numpy as np

#------------------------------ Classical Gram-Schmidt (CGS)

def cgs(A):
    '''
    Performs QR decomposition using Classical Gram-Schmidt method.

    Parameters:
     A : numpy.ndarray, Input matrix of size (m x n)

    Returns:
     Q : numpy.ndarray, Orthonormal matrix (m x n)
     R : numpy.ndarray, Upper triangular matrix (n x n)
    '''
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
```

```
    for j in range(n):
        v = A[:, j].copy()   # v_j = a_j

        for i in range(j):
            R[i, j] = np.dot(Q[:, i], A[:, j])
            v = v - R[i, j] * Q[:, i]

        R[j, j] = np.linalg.norm(v)
        Q[:, j] = v / R[j, j]

    return Q, R


#------------------------------ Modified Gram-Schmidt (MGS)

def mgs(A):
    '''
    Performs QR decomposition using Modified Gram-Schmidt method.

    Parameters:
     A : numpy.ndarray, Input matrix of size (m x n)

    Returns:
     Q : numpy.ndarray, Orthonormal matrix (m x n)
     R : numpy.ndarray, Upper triangular matrix (n x n)
    '''
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    V = A.copy()

    for i in range(n):
        R[i, i] = np.linalg.norm(V[:, i])
        Q[:, i] = V[:, i] / R[i, i]

        for j in range(i + 1, n):
            R[i, j] = np.dot(Q[:, i], V[:, j])
            V[:, j] = V[:, j] - R[i, j] * Q[:, i]

    return Q, R


#------------------------------ Orthogonality Error

def orthogonality_error(Q):
    '''
    Computes the orthogonality error of Q.
```

```
    Formula: ||I - Q^T Q||_F

    Parameters:
     Q : numpy.ndarray, Orthonormal matrix

    Returns:
     float, Frobenius norm error
    '''
    n = Q.shape[1]
    I = np.eye(n)
    return np.linalg.norm(I - Q.T @ Q, ord='fro')


#------------------------------ Ill-conditioned Matrix

def create_matrix(eps=1e-8):
    '''
    Generates a nearly linearly dependent matrix.

    Parameters:
     eps : float, small perturbation value

    Returns:
     A : numpy.ndarray, ill-conditioned matrix
    '''
    A = np.array([
        [1, 1, 1],
        [1, 1, 1 + eps],
        [1, 1 + eps, 1]
    ], dtype=float)

    return A


#------------------------------ Random Matrix Generator

def generate_matrix(m=200, n=100):
    '''
    Generates a random matrix with normal distribution.

    Parameters:
     m : int, number of rows
     n : int, number of columns

    Returns:
     numpy.ndarray, random matrix
```

```
    '''
    np.random.seed(0)  # reproducibility
    return np.random.randn(m, n)


#------------------------------ Result Printer

def print_results(name, Q, R, err):
    '''
    Prints formatted QR decomposition results.

    Parameters:
     name : str, method name (CGS / MGS)
     Q : numpy.ndarray, orthonormal matrix
     R : numpy.ndarray, upper triangular matrix
     err : float, orthogonality error
    '''
    print("-" * 50)
    print(f"{name} RESULTS")
    print("-" * 50)

    print("\nQ matrix:")
    print(np.round(Q, 6))

    print("\nR matrix:")
    print(np.round(R, 6))

    print("\nOrthogonality Error ||I - Q^T Q||_F:")
    print(f"{err:.6e}")
    print("\n")
```

1.1 Execution

[38]:
```
A = create_matrix()
print(A)
```

```
[[1.         1.         1.        ]
 [1.         1.         1.00000001]
 [1.         1.00000001 1.        ]]
```

[39]:
```
# CLASSICAL GRAM-SCHMIDT (CGS)
Q_cgs, R_cgs = cgs(A)
err_cgs = orthogonality_error(Q_cgs)

# MODIFIED GRAM-SCHMIDT (MGS)
Q_mgs, R_mgs = mgs(A)
err_mgs = orthogonality_error(Q_mgs)
print_results("CLASSICAL GRAM-SCHMIDT (CGS)", Q_cgs, R_cgs, err_cgs)
```

```
print("-"*50)

print_results("MODIFIED GRAM-SCHMIDT (MGS)", Q_mgs, R_mgs, err_mgs)
print("-"*50)
```

```
--------------------------------------------------
CLASSICAL GRAM-SCHMIDT (CGS) RESULTS
--------------------------------------------------

Q matrix:
[[ 0.57735  -0.408248 -0.453256]
 [ 0.57735  -0.408248 -0.361521]
 [ 0.57735   0.816497  0.814777]]

R matrix:
[[ 1.732051  1.732051  1.732051]
 [ 0.        0.       -0.      ]
 [ 0.        0.        0.      ]]

Orthogonality Error ||I - Q^T Q||_F:
1.411235e+00


--------------------------------------------------
--------------------------------------------------
MODIFIED GRAM-SCHMIDT (MGS) RESULTS
--------------------------------------------------

Q matrix:
[[ 0.57735  -0.408248 -0.707107]
 [ 0.57735  -0.408248  0.707107]
 [ 0.57735   0.816497 -0.      ]]

R matrix:
[[ 1.732051  1.732051  1.732051]
 [ 0.        0.       -0.      ]
 [ 0.        0.        0.      ]]

Orthogonality Error ||I - Q^T Q||_F:
1.776357e-07


--------------------------------------------------
```

[40]:
```
# Final comparison
print("-"*50)
print("COMPARISON")
```

```python
print("-"*50)
print(f"CGS Error : {err_cgs:.6e}")
print(f"MGS Error : {err_mgs:.6e}")
print("-"*50)

if err_mgs < err_cgs:
    print("\n MGS preserves orthogonality better than CGS.")
else:
    print("\n Unexpected result.")
print("-"*50)
```

```
--------------------------------------------------
COMPARISON
--------------------------------------------------
CGS Error : 1.411235e+00
MGS Error : 1.776357e-07
--------------------------------------------------

 MGS preserves orthogonality better than CGS.
--------------------------------------------------
```

1.2 Question 4(e): Comparison of CGS and MGS

Based on the numerical execution for the input matrix A, we observe a significant difference in the stability of the two orthogonalization methods.

The results show that the **Modified Gram–Schmidt (MGS)** method preserves orthogonality much better than the **Classical Gram–Schmidt (CGS)** method.

- **In CGS**, the orthogonality error is very large (≈ 1.411), which indicates a near-total loss of orthogonality. This suggests that the computed Q matrix is not close to being orthonormal, essentially "collapsing" due to the near-linear dependence of the columns in A.
- **In MGS**, the error is maintained at a very small magnitude ($\approx 1.776 \times 10^{-7}$). This demonstrates that MGS is numerically stable enough to handle the precision required for this matrix.

This discrepancy occurs because of how rounding errors propagate: 1. **CGS (Classical):** Projections are calculated using the original vectors. When columns are nearly parallel (as in our input matrix A), the subtraction of large, nearly equal numbers leads to catastrophic cancellation. The error accumulates rapidly because each step relies on the original, "uncleaned" vectors. 2. **MGS (Modified):** Each vector is orthogonalized against the *already updated* versions of the remaining vectors. This sequential "correction" step ensures that rounding errors from previous projections are partially accounted for in subsequent steps, significantly reducing error buildup.

Conclusion: MGS is the superior choice for practical computations where the input matrix is ill-conditioned or nearly singular.

1.3 ——————

2 Question 5

Generate a random matrix

$$A \in \mathbb{R}^{200\times 100}.$$

(a) Compute a QR factorization using your implementation of Modified Gram–Schmidt.

(b) Compute QR using the built-in routine in Python.

(c) Compare the orthogonality error

$$\|I - Q^T Q\|.$$

(d) Solve the least squares problem

$$Ax = b$$

for a random vector b.

(e) Compare the residual norms $\|Ax - b\|$ and comment briefly on the numerical behavior you observe. behavior you observe.

[41]:
```
A1 = generate_matrix()

Q, R = mgs(A1)
print("MODIFIED GRAM-SCHMIDT QR (MGS)", Q, R)
Q_np, R_np = np.linalg.qr(A1)
print("NUMPY QR", Q_np, R_np)
```

```
MODIFIED GRAM-SCHMIDT QR (MGS) [[ 0.12083972  0.02259065  0.0700394  …
0.04361943  0.02897362
  -0.05003432]
 [ 0.1289981  -0.10832973 -0.11184511 …  0.10086637  0.10107054
   0.03361919]
 [-0.0252894  -0.01634121  0.0807662  …  0.00724177  0.07398725
   0.0910152 ]
 …
 [-0.06681267  0.02033478  0.01649092 …  0.0283551  -0.14611121
   0.13890391]
 [ 0.14984134  0.02867344 -0.03334094 … -0.02173754 -0.06552059
  -0.11654869]
 [ 0.02160014 -0.02878867 -0.0301228  … -0.03984685 -0.06355371
  -0.0450948 ]] [[14.598282    0.80614938  0.56729027 …  1.32039809
2.48095336
   1.1187632 ]
 [ 0.         13.40122261 -1.51965186 … -0.61243472  0.05885099
   0.92942525]
 [ 0.          0.         13.48550534 … -0.11109094 -1.98762979
   0.44827272]
```

```
 …
 [ 0.          0.          0.          …  8.44191717 -1.45379036
  -0.15470123]
 [ 0.          0.          0.          …  0.         10.43399815
   0.14967676]
 [ 0.          0.          0.          …  0.          0.
   9.67741989]]
NUMPY QR [[-0.12083972  0.02259065 -0.0700394  … -0.04361943 -0.02897362
   0.05003432]
 [-0.1289981  -0.10832973  0.11184511 … -0.10086637 -0.10107054
  -0.03361919]
 [ 0.0252894  -0.01634121 -0.0807662  … -0.00724177 -0.07398725
  -0.0910152 ]
 …
 [ 0.06681267  0.02033478 -0.01649092 … -0.0283551   0.14611121
  -0.13890391]
 [-0.14984134  0.02867344  0.03334094 …  0.02173754  0.06552059
   0.11654869]
 [-0.02160014 -0.02878867  0.0301228  …  0.03984685  0.06355371
   0.0450948 ]] [[-14.598282    -0.80614938  -0.56729027 …  -1.32039809
-2.48095336
   -1.1187632 ]
 [  0.          13.40122261  -1.51965186 …  -0.61243472   0.05885099
   0.92942525]
 [  0.           0.         -13.48550534 …   0.11109094   1.98762979
   -0.44827272]
 …
 [  0.           0.           0.          …  -8.44191717   1.45379036
   0.15470123]
 [  0.           0.           0.          …   0.         -10.43399815
   -0.14967676]
 [  0.           0.           0.          …   0.           0.
   -9.67741989]]
```

[42]:
```
orth_error_mgr = orthogonality_error(Q)
orth_error_np = orthogonality_error(Q_np)
print("-"*50)
print(f"{'Method':<20} {'Orthogonality Error':<25} ")
print("-"*50)

print(f"{'MGS':<20} {orth_error_mgr:.6e} {'':<5} ")
print(f"{'NumPy QR':<20} {orth_error_np:.6e} {'':<5} ")

print("-"*50)
```

```
--------------------------------------------------
Method               Orthogonality Error
--------------------------------------------------
```

```
MGS                     6.856215e-15
NumPy QR                7.310522e-15
--------------------------------------------------
```

[43]:
```
b = np.random.randn(A1.shape[0])
Qt_b = Q.T @ b
xmgs = np.linalg.solve(R, Qt_b)
residual_mgs = np.linalg.norm(A1@xmgs - b)
Qt_b_np = Q_np.T @ b
xnp = np.linalg.solve(R_np, Qt_b_np)
residual_np = np.linalg.norm(A1@xnp - b)
print("-"*50)

print("\nSolution (MGS):")
print(xmgs)
print("-"*50)

print("\nSolution (NumPy):")
print(xnp)
print("-"*50)
```

```
--------------------------------------------------

Solution (MGS):
[ 0.0371378  -0.07739965 -0.04259685 -0.08778465  0.00396255 -0.10242261
 -0.04855712  0.06903126  0.04440786  0.12584478  0.18656438  0.06658996
  0.02379105  0.00547012  0.01413943 -0.06261587  0.10687284 -0.07607201
  0.00668366 -0.0598029  -0.12102076  0.09419653  0.17125187  0.07876864
 -0.06867557 -0.04430468  0.12140609  0.0582947  -0.10188815 -0.04104981
 -0.04686297  0.11443639  0.13933503  0.00372111 -0.00071506  0.09853706
 -0.12952765 -0.03625936 -0.12060323 -0.06128811  0.07704444 -0.19071793
  0.02147021 -0.12669911 -0.07941415  0.12744906  0.0406655  -0.13891833
  0.02790918  0.12986029 -0.00130171 -0.09161296 -0.08129596 -0.07429951
  0.13232384  0.00585589  0.1131096  -0.13652856 -0.01447371  0.01397884
 -0.14772882  0.07808897 -0.02202257 -0.04418866 -0.15256513 -0.03700364
  0.05474883 -0.08763441 -0.09777066 -0.01565643  0.00342654 -0.0534748
 -0.07967606  0.05887877 -0.14143281 -0.16900958 -0.12614626 -0.14486019
 -0.04588975 -0.00649991  0.12135609 -0.12523647 -0.05640688 -0.05548547
 -0.1091986  -0.15067693  0.04455736  0.05003692 -0.12260026  0.02880577
 -0.06383271 -0.00538312  0.01065899 -0.08428429  0.02400554 -0.0603535
 -0.02793105  0.0739107  -0.02382809 -0.10436055]
--------------------------------------------------

Solution (NumPy):
[ 0.0371378  -0.07739965 -0.04259685 -0.08778465  0.00396255 -0.10242261
 -0.04855712  0.06903126  0.04440786  0.12584478  0.18656438  0.06658996
  0.02379105  0.00547012  0.01413943 -0.06261587  0.10687284 -0.07607201
  0.00668366 -0.0598029  -0.12102076  0.09419653  0.17125187  0.07876864
```

```
 -0.06867557 -0.04430468  0.12140609  0.0582947  -0.10188815 -0.04104981
 -0.04686297  0.11443639  0.13933503  0.00372111 -0.00071506  0.09853706
 -0.12952765 -0.03625936 -0.12060323 -0.06128811  0.07704444 -0.19071793
  0.02147021 -0.12669911 -0.07941415  0.12744906  0.0406655  -0.13891833
  0.02790918  0.12986029 -0.00130171 -0.09161296 -0.08129596 -0.07429951
  0.13232384  0.00585589  0.1131096  -0.13652856 -0.01447371  0.01397884
 -0.14772882  0.07808897 -0.02202257 -0.04418866 -0.15256513 -0.03700364
  0.05474883 -0.08763441 -0.09777066 -0.01565643  0.00342654 -0.0534748
 -0.07967606  0.05887877 -0.14143281 -0.16900958 -0.12614626 -0.14486019
 -0.04588975 -0.00649991  0.12135609 -0.12523647 -0.05640688 -0.05548547
 -0.1091986  -0.15067693  0.04455736  0.05003692 -0.12260026  0.02880577
 -0.06383271 -0.00538312  0.01065899 -0.08428429  0.02400554 -0.0603535
 -0.02793105  0.0739107  -0.02382809 -0.10436055]
--------------------------------------------------
```

[44]:
```
diff = np.linalg.norm(xmgs - xnp)
print("-"*50)
print("||x_mgs - x_numpy|| =", diff)
print("-"*50)
```

```
--------------------------------------------------
||x_mgs - x_numpy|| = 1.0858984026382148e-15
--------------------------------------------------
```

[45]:
```
print("-"*50)
print(f"{'Method':<20} {'Residual ||Ax-b||':<25} ")
print("-"*50)

print(f"{'MGS':<20} {residual_mgs:.6e} {'':<6} ")
print(f"{'NumPy QR':<20} {residual_np:.6e} {'':<6} ")

print("-"*50)
```

```
--------------------------------------------------
Method               Residual ||Ax-b||
--------------------------------------------------
MGS                  1.051780e+01
NumPy QR             1.051780e+01
--------------------------------------------------
```

2.1 Question 5 (e)

The residuals for both MGS and NumPy QR are almost exactly the same. This means both methods are giving the same solution to the least squares problem.

This happens because the matrix we used is well-conditioned, so even a slightly less stable method like MGS still works very well.

Although NumPy's method is generally more stable, in this case the difference doesn't really matter both perform equally well.